AIStudio

AIStudio Tutorial

Version: 1.0.0 · Updated 2026-06-14

Get the most out of AIStudio with four guided modules — from your first query, to two production-scale corpora, to your own documents.

> ***Prerequisites:*** *AIStudio must be installed and running. If you haven't done that yet, start with QUICKSTART.md.*
>
> *Sections marked Going deeper explain the **why** behind the **how**. Skip them on a first pass — the steps work without them. For the full mechanics behind a step, the **Annexes** carry the in-the-weeds reality the modules deliberately leave out.*
>
> ***Radical transparency.*** *AIStudio is built so you never have to take its word for anything. Every claim in every answer carries a clickable citation to the exact source and page — verify it in two clicks. It also doesn't hide that the answer depends on the **model**: ask the same question of two models and the emphasis shifts (§2.5 shows this side by side), and the citations are how you tell which framing the documents actually support. And where the happy path ends — the pre-verified data and default settings Modules 2–3 walk — the body says so and points you to the Annex that carries the messier truth. The annexes are the payoff, not an apology. The tool's job is not to sound authoritative; it is to make its reasoning checkable.*

Module 1 — Quick Tour

Goal: Run your first query, understand citations, explore the interface. ~15 minutes.

1.1 Your First Query

AIStudio ships with a **demo** corpus — original documents spanning enterprise architecture, IT strategy, financial-services technology, and agentic AI. It's already indexed and ready.

Open AIStudio:

```
ais_start
```

When AIStudio opens, the **left sidebar** is your control panel. The **CORPUS** section near the top is where you choose which document set to query — **demo** is pre-selected. **New**, **Delete**, and **Rename** do what they say; the **Edit** button (corpus settings) we cover in §4.6. Below them, two buttons manage a corpus's files: **Add** ingests a new file into the selected corpus, and **Inspect** lists the corpus's files so you can remove or re-index them.

We'll build a short **three-question thread** across §1.1–§1.3 that walks the whole loop — grounding, then conversation memory, then multi-source citations. Start with the opener:

- *"What is the root of the word strategy?"* ← **start here**

The answer traces *strategy* to the Greek στρατηγία ("generalship"), cited to a specific page of the source — your first look at a grounded, attributable answer.

> ***Tip — recall a question.*** *Press ↑ / ↓ in the question box to scroll back through questions you've already asked this session, edit, and re-send — no retyping.*

1.2 Understanding Citations

Every answer carries citations — `[1]`, `[2]` — and a **References** panel showing exactly which document and page each claim came from. **The inline `[N]` markers are clickable**: click one — or the **Open** ↗ on its References entry — to open the source in the **Source Dive** panel at the cited page (PDFs); other file types open in a new tab. This is AIStudio's core promise: answers grounded in your documents, with a path back to the source.

1.3 Follow-up Questions

AIStudio keeps conversation context across a session, so you can build on the previous answer with pronouns — it remembers what *that* and *this* refer to. Continue the thread from §1.1:

- *"How does that differ from tactics?"* — *that* = strategy, from your first question; the answer stays grounded in the same paper.
- *"How does this apply to the development of an Enterprise Architecture?"* — this one draws on several documents at once; watch the **References** panel fill with multiple sources, each claim carrying its own clickable `[N]`.

1.4 Tuning Your Query

In the **Query Settings** section of the sidebar — just below the CORPUS area — are the knobs that shape how AIStudio retrieves and answers. Let's walk each one.

Top K is how many document chunks AIStudio retrieves to answer from: - Default 5 chunks. **Use 10 for the demo** — it includes small documents (the QFD paper is 34 chunks) that get outcompeted at K=5 and surface reliably at K=10. - **Use 10 for SEC 10-K and ESEF** — these are the large financial-filing corpora you'll build in Modules 2 and 3 (annual reports from many firms). There you'll ask cross-firm *comparison* questions, which need chunks from several filings at once. AIStudio also raises Top K automatically when a query names several firms (~2 more chunks per firm, ceiling K=20).

Temperature controls creativity (0.1–0.3 precise, 0.5–0.7 more synthesis; default 0.3 suits document Q&A).

Retrieval Mix is a slider that runs **left → right = literal → conceptual**. Drag **left** for literal matching — exact terms, tickers, defined phrases, names; drag **right** for conceptual matching — themes and ideas where the wording varies; the **middle (default) blends both**. (Under the hood the slider blends keyword/BM25 and semantic/vector retrieval; the panel shows it in plain literal↔conceptual terms.)

Score Threshold is a relevance floor — retrieved chunks scoring below it are dropped before the model sees them, which suppresses weak, off-topic context. AIStudio stores a per-corpus default (demo 0.3, SEC 10-K 0.5). **When to lower it:** the embedding model (`nomic-embed-text`) often scores genuinely-relevant chunks below 0.5, so if good answers come back hedged or empty, drop the floor toward **~0.2–0.3** and re-ask.

1.5 The Help Corpus — AIStudio Answering About Itself

Switch to the **help** corpus and ask:

- *"How do I re-ingest a corpus?"*
- *"What embedding model does AIStudio use?"*

Same RAG pipeline, pointed at AIStudio's own docs. Try it before reaching for the manual.

Module 2 — At Scale: The SEC 10-K Corpus

Goal: Build a large corpus from scratch, give it the reference data it needs to retrieve well, and query it. ~45 minutes (30 min ingest).

This module downloads a portfolio of SEC 10-K annual filings, ingests them, and queries them. It also introduces the part of AIStudio that makes retrieval at this scale work: **knowledge bases**. The corpus ships with a benchmark question set; running it and — more importantly — reading it correctly is **Annex 4**.

2.1 Download the Filings

```
ais_download_sec_10k
```

This reads the corpus's **membership list** (a manifest) and pulls each firm's most recent 10-K filings from SEC EDGAR into the corpus. Allow 5–10 minutes.

EDGAR serves filings by **CIK** — the *Central Index Key*, the unique number the SEC assigns to every entity that files with it (look one up at the SEC's CIK lookup). So preparing this corpus came down to building a small **manifest** — one row per firm, pairing a **Label** (the name AIStudio uses for the firm throughout the system) with two identifiers: its **CIK**, the key EDGAR needs to return the filings, and a verified **LEI** (the 20-character global Legal Entity Identifier), which §2.2 puts to work. In short: a manifest of CIKs and LEIs. Its full structure is in **Annex 1**.

> ***What is SEC EDGAR?*** *The SEC requires public companies to file annual reports (10-K); EDGAR is the public database where they live. AIStudio's downloader handles the access protocol — you get the files, it handles the plumbing.*
>
> ***Firms included:*** *Goldman Sachs, JPMorgan Chase, Morgan Stanley, BlackRock, BNY Mellon, Citigroup, and others — bulge-bracket banks, asset managers, exchanges, custody banks.*

Going deeper — Why these filings need more than raw text

(skippable)

In Module 1 you queried the **demo** corpus — and if you select it and click **Inspect**, you'll see it's nothing but plain PDFs dropped straight in (Module 4 walks you through building a corpus that way from your own PDF or text files). General documents like these are written *for* a reader — topic, dates, subject are all in plain prose.

Filings are different: hundreds of pages of tables, footnotes, and boilerplate. Even *"what year is this filing for?"* is hard to answer reliably from raw text — you'd need a brittle heuristic that breaks on the next corpus. We want something corpus-independent.

The companies already solved this at the source. Filings aren't loose text — they're filed in a standardized, machine-readable format: **iXBRL (inline XBRL)**, an HTML document with embedded tags that codify the key facts, including **which entity** the filing covers and **what period**. So the first thing AIStudio does at ingestion is read those tags directly — pulling the company name and fiscal year with no reliance on the filename.

2.2 Recognizing Unique Company Names

AIStudio reads two iXBRL tags from each filing:

- `dei:EntityRegistrantName` — the company the filing is about
- `dei:DocumentFiscalYearFocus` — the SEC-mandated fiscal year

That anchors *what this document is* — but it isn't enough. The same firm appears as *"JPMorgan"*, *"JPMorgan Chase"*, *"JPM"*, or a ticker, while the filing's registered name is `JPMORGAN CHASE & CO.` Retrieval has to know these are the **same entity**.

AIStudio handles this with an **entity knowledge base**: a mapping from each firm's canonical legal name to its natural variants. The canonical name and its aliases are pulled from **GLEIF** (the Global Legal Entity Identifier Foundation — the authoritative open registry of legal entities) **by LEI**, then enriched with short names and tickers.

The crucial design point is *how* a firm is looked up. You might expect a search by company name — but legal names are ambiguous: a multinational has dozens of subsidiaries, depositary receipts, and namesakes, and a name search confidently returns the *wrong* entity often enough to corrupt a corpus silently. So AIStudio resolves each firm by its **LEI** — the 20-character Legal Entity Identifier that names exactly one entity worldwide. **The LEI is the input, not a guess:** the corpus ships with a verified LEI for every firm.

Build the entity KB:

```
ais_import_entity_kb --corpus sec_10k --apply
```

This scans the downloaded filings, resolves each firm by its verified LEI against GLEIF to pull the canonical name and aliases, and writes the **entity knowledge base** — keyed by the name each filing reports for itself, carrying that firm's canonical name, LEI, and aliases. (Its exact structure and where it's stored are in **Annex 1**.)

> *For the shipping portfolio every firm resolves cleanly on its verified LEI, so this is one command. When you build entity support for your **own** corpus (Module 4), you supply the LEIs — and if a firm doesn't resolve, the command pauses and hands you a worksheet to confirm it. **Annex 1** is the full story: what goes wrong when a name search guesses, and the one-time review gate that catches it.*

Going deeper — How a chunk knows which company it's about

(skippable)

At ingestion AIStudio splits each filing into many small **chunks** and embeds each one. A query is embedded the same way, and retrieval matches question to chunks. The problem at scale: a chunk of dense financial text, lifted out of a 200-page filing, often carries no clue which firm it came from — a paragraph on capital ratios reads almost identically across twenty-five banks.

AIStudio fixes this with **chunk enrichment**: at ingestion, every chunk is prefixed with a small label carrying the company name, its aliases, and the filing year — drawn from the entity KB and the iXBRL tags:

```
[Document: THE GOLDMAN SACHS GROUP, INC. | Goldman Sachs | GS FY2025] <chunk text…>
```

Now identity travels *with* the text. When your question names a firm, retrieval can restrict to that firm's chunks — and because the label carries aliases, it doesn't matter whether you typed "Goldman", "Goldman Sachs", or "GS".

2.3 Industry Terms — the Glossary

Company names aren't the only vocabulary gap. Ask about *"CET1"* and the passage may only ever say *"Common Equity Tier 1 capital ratio"*, or vice versa.

AIStudio bridges this with a second knowledge base — a **glossary** of domain terms and expansions, sourced for the bank corpora from the **BIS Basel** framework. It ships ready-built. (Its structure and storage are in **Annex 2**.)

Build (or refresh) it with:

```
ais_import_glossary_kb --source bis_basel
```

Unlike the entity KB, the glossary is **corpus-wide** — "CET1" means the same thing everywhere — and it needs no review gate (see Annex 2).

> ***Before you ingest:*** *the entity KB (§2.2) must exist at ingestion time — chunk enrichment depends on it. The glossary needs nothing built at ingestion; it's read later, at query time.*

Going deeper — When the glossary is applied (query time, not ingestion)

(skippable)

An asymmetry worth understanding. The **entity** label is baked into each chunk *at ingestion* — it becomes part of the stored text. The **glossary** works the other way: chunks are **not** rewritten; the mapping is applied to **your question, at query time**. Ask about *"CET1"* and AIStudio widens the search to include *"Common Equity Tier 1 capital ratio …"* before retrieving — so an acronym query finds a spelled-out passage and vice versa, with the stored chunks left untouched. This is

the spine of the two reference layers: entity = build-time + query-filter, glossary = query-time expansion.

2.4 Ingest the Corpus

With the entity KB built and AIStudio running:

```
ais_ingest_sec_10k
```

~30 minutes on an M4 MacBook Pro. This is where chunk enrichment happens — as each filing is chunked, AIStudio reads its iXBRL entity and year and prefixes every chunk with the `[Document: …]` label. The summary reports how many files were augmented and which entities it recognized, and closes with a `· File Ingested:` roster of every file written this run.

> *Leave the terminal open; ingestion is CPU- and memory-intensive.*

2.5 Query at Scale

Switch to the **sec_10k** corpus. First, a word on **model choice**. As covered in QUICKSTART, AIStudio runs its models locally through Ollama, and you can add as many as you like — some small and fast, some large. For the easy questions in Module 1 the difference between a small and a large model is negligible, in both answer quality and speed. That changes here: SEC filings are dense, and questions that span several firms or turn on a precise disclosure reward a model with stronger synthesis and tighter citation discipline — a small model starts to trade accuracy for speed (Annex 4 quantifies the cost).

So for this module, **select `gemma3:27b` in the model dropdown** — Google's flagship open model (Gemma 3, 27 billion parameters), and the synthesis model AIStudio is tuned around. Because it's a large model it needs a reasonably capable machine to run at good speed; if you don't have it yet, pull it once through Ollama (QUICKSTART shows how to add a model). Start with the showcase question:

- *"Which financial firms have dedicated AI governance committees?"* ← **showcase** (cross-firm, answered from prose disclosures)

Then:

- *"What does Goldman Sachs say about the risks of artificial intelligence?"*
- *"How does JPMorgan Chase describe their cybersecurity risk management?"*

> ***Tip:*** *Use the* ***Firm*** *filter to restrict retrieval to one firm (type* `Goldman Sachs`*); leave blank for cross-corpus queries.*
>
> ***Try a few models.*** *Once you've added more than one model (QUICKSTART covers it), re-ask the same question on each from the model dropdown and compare — a small model answers markedly faster, a large one more thoroughly and with tighter citations. Switching models keeps your conversation: AIStudio shows a brief "Changed to <model>" line, and your ↑ / ↓ question history still works across the switch, so you can recall the previous question and send it to the new model without retyping.*
>
> ***Switching corpora.*** *Whenever you change the selected corpus, AIStudio prints a short "— Changed to <corpus> —" line in the chat, so you always know which document set the next answer will draw on.*
>
> ***Worked example — the same question, two models.*** *Ask "What does Goldman Sachs say about the risks of artificial intelligence?" on* ***gemma3:27b****, then switch the model to* ***mistral:7b*** *and press ↑ then Enter to re-ask. Both answers are grounded and cite real Goldman filings — but they differ. The larger model is more exhaustive: it surfaces an extra year's filing and names the governance committees that oversee AI risk. The smaller model is tighter and faster, covering the same core (regulatory uncertainty, model error and bias, third-party reliance, bad-actor misuse) in fewer words. Neither is "wrong." The model sets the depth and emphasis; the citations are how you confirm which framing the filings actually support. That is the radical-transparency note from the top of this tutorial made concrete — read the sources, don't take the wording on faith.*
>
> ***When the answer is a number from a table, read it twice.*** *A cross-firm numeric query — "Compare the CET1 ratios for JPMorgan and Citigroup" — is where AIStudio is most likely to return a confident wrong figure: financial ratios live in multi-year, multi-column tables, and a chunk can sever a cell from the column header (the year) that gives it meaning.* ***Annex 5*** *is the full worked case — one of those two firms comes back exactly right and the other a full year off, and the contrast is the whole lesson.*

2.6 Add a Firm on Demand

You don't have to rebuild the whole corpus to add one company. The flow is **download** → **selective re-ingest** → **query**, and it leaves the other ~100 filings untouched.

1. Download just the new firm (by ticker — download stays a manual step):

```
ais_download_sec_10k --tkr BLK --latest
```

> ***BlackRock dual-CIK.*** *BlackRock reorganized in 2024 and received a new EDGAR CIK (`0002012383`). The ticker `BLK` resolves to this new CIK, which only carries FY2024–FY2025. To build a full 5-year history, supplement with the legacy CIK:*
>
> ```*bash*
>
> # *FY2024–FY2025 (new CIK 0002012383)*
>
> *ais_download_sec_10k --tkr BLK --latest*
>
> # *FY2021–FY2023 (legacy CIK 0000014272)*
>
> *ais_download_sec_10k --cik 0000014272 --force_name "BlackRock" --latest 3*
>
> # *Ingest both*
>
> *ais_ingest_sec_10k --files BlackRock* ```
>
> *This pattern applies to any firm whose EDGAR CIK changed through a reorganization.*

2. Re-ingest only that file. `ais_ingest_sec_10k` is incremental by default (it skips unchanged files), so a plain run already picks up the new filing. But you can also name exactly what to ingest with `--files`:

```
ais_ingest_sec_10k --files BlackRock
```

`--files` takes one or more comma-separated **patterns**, OR-matched against the filename: each is a literal substring, or a regex if it contains regex metacharacters (`* + ? [ ] ( ) | ^ $ { } \`). So `--files BlackRock` matches `BlackRock_10K_2025-02-25.htm`; `--files 'JPM.*2025,Citi'` matches either. Everything not matched is left untouched, and the run ends with a `· File Ingested:` roster listing exactly what landed.

3. Query it. Switch to **sec_10k**, set the **Firm** filter to `BlackRock, Inc.` (or leave it on auto — the entity KB resolves the filter), and ask. BlackRock answers end-to-end the moment its filing is ingested: plain ingest stamps the registrant tag that drives firm isolation, so no separate enrichment step is required.

> ***Correcting the firm's identity.*** *An on-demand add resolves its LEI by name-search — a machine guess until a human verifies it. Annex 1 §A1.5 walks the one-edit correction, with BlackRock as the worked case.*

Module 3 — A Second Corpus: European Banks (ESEF)

Goal: Build a second production corpus that mirrors Module 2 — same four steps, a different regulator and access key. ~40 minutes.

The SEC corpus is US filers retrieved from EDGAR by **CIK**. European banks file under the EU's **ESEF** mandate, retrieved from **filings.xbrl.org** by **LEI**. The shape of the work is identical — download → entities → glossary → ingest — which is the point: the corpus machinery is source-independent, only the access key and the source endpoint change.

3.1 Download the Filings

```
ais_download_esef
```

This reads the corpus's scope file (`esef_banks_full_scope.yaml`) — one row per bank, keyed by LEI — and queries filings.xbrl.org by LEI for each bank's ESEF annual reports, into `data/corpora/esef_banks/uploads/` .

> ***Filing years may differ across firms.*** `ais_download_esef` *fetches the latest available filing for each firm — but "latest available" is not always the same fiscal year for every firm. Some banks file months after the calendar year end; others have not yet published. In a corpus built in mid-2026, BBVA and SEB were at FY2024 while the other EN-primary firms were at FY2025. Check the download summary and each firm's year in the* `--- Downloading` *output before running cross-firm year-over-year queries.*
>
> ***Why LEI here and CIK for SEC?*** *Each regulator exposes a different access key — EDGAR indexes by CIK, filings.xbrl.org filters by LEI. AIStudio keeps the source specifics (endpoint, access key) as data, not code, so adding a regulator is a configuration change, not a rewrite. The* ***LEI is still the identity*** *in both — see Annex 1.*

3.2 Recognizing Unique Company Names

```
ais_import_entity_kb --corpus esef_banks --apply
```

Same command as SEC, pointed at the European corpus: it scans the ESEF filings, resolves each bank by its verified LEI, and writes `data/knowledge_sources/gleif/esef_banks_full_entities.yaml`.

> ***This is where Europe gets interesting.*** *A name search for these banks fails far more often than for US filers — subsidiaries, depositary receipts, and US-jurisdiction defaults mismatch firms like Société Générale, Crédit Agricole, and BBVA. That's exactly why the LEI is the input and not a guess. On the shipping portfolio the verified LEIs make this one clean command; the failure modes — and the one-time worksheet that catches them — are* ***Annex 1****, and the deeper "why non-English filings are harder" is* ***Annex 3****.*

3.3 Industry Terms — the Glossary

```
ais_import_glossary_kb --source bis_basel
```

The same BIS Basel glossary applies — capital and regulatory terms are shared vocabulary across US and European banks. The glossary is corpus-wide; there is nothing ESEF-specific to build.

3.4 Ingest the Corpus

```
ais_ingest_esef
```

Same enrichment as SEC: each chunk is prefixed `[Document: <entity> | <aliases> FY<year>]` from the entity KB and the iXBRL tags.

3.5 Query at Scale

Switch to the **esef_banks** corpus (again `gemma3:27b` — a larger model buys real multilingual headroom here, see Annex 3):

- *"How does BNP Paribas describe its CET1 capital position?"* ← **BIS / Basel-glossary example**
- *"What do European banks say about climate-related financial risk?"*

Going deeper — When the portfolio isn't in English

(skippable)

A US-only corpus hides a problem that a European one surfaces immediately: retrieval quality is not language-neutral. A question in English against a filing that discusses *fonds propres de base*

de catégorie 1 instead of *Common Equity Tier 1* retrieves worse, and a firm whose filing language is mislabeled retrieves worse still. The glossary helps with terminology, but it isn't the whole story. **Annex 3** is the full account of what degrades when the portfolio leaves English.

Module 4 — Bring Your Own Corpus

Goal: Ingest your own documents and query them. ~15 minutes + ingest.

Modules 2 and 3 used the operator seed machinery that ships the SEC and European corpora. Your own corpus uses the **UI** — you build directly what those corpora arrive as at runtime.

4.1 Create a New Corpus

In the browser: **Settings** → **New Corpus**, name it (e.g. `my_docs`), **Create**. This makes `data/corpora/my_docs/` with an `uploads/` folder.

4.2 Upload Your Documents

With the corpus selected, click **Add** and choose files. Supported: PDF (page-aware), DOCX, PPTX, XLSX, Markdown, HTML. AIStudio ingests automatically after upload and shows progress in the chat area.

4.3 Query Your Corpus

Select your corpus and ask. Same interface, your content.

4.4 Optional — Entity support for your own corpus

If your documents are entity-centric (filings, reports about specific firms), you can give your corpus the same entity KB the shipped corpora use — but **you supply the verified LEIs**, because there's no pre-built seed for a user corpus. That is precisely the workflow **Annex 1** documents end to end: scan, review the worksheet, fill the LEI where the guess is wrong, apply.

4.5 Optional — Write Your Own Benchmark Questions

Create `benchmarks/<your-corpus>/<your-corpus>_questions.yaml`:

```
- topic: Topic Name
  questions:
    - id: unique_id
      question: The exact question sent to AIStudio.
```

```
    keywords: [term1, term2, term3]   # all must appear in the answer to pass
    notes: What a correct answer looks like.
```

Then `ais_bench --corpus your-corpus-name`. What the pass/fail checks really mean is **Annex 4**.

4.6 Optional — Add Corpus Guidance

Use the **Edit Corpus** modal to set description, summary, and search guidance (routing hints that tell the model which document to consult for which question) — written straight to the runtime `<your-corpus>_corpus_metadata.yaml`. No YAML editing required.

Annexes

The modules walk the happy path. What follows is deliberately a mix of three different kinds of thing, and it helps to know which one you're reading at any moment:

- ***How the system is actually structured*** *— the weeds. How the shipped corpora were really built, where each piece lives, and how the parts fit together. This is not an architecture description. For the general picture — the components, the objects, and especially the data and how it flows — read the companion note AIStudio — Elements of an Architecture.*
- ***Lessons learned building it*** *— findings from the problem space itself, including the parts that fight back: pulling numbers out of dense tables, and handling filings that aren't in English.*
- ***The shape of how we evaluate*** *— the broad contour of the method we use to measure results continuously. Getting this right was a journey, not a one-time setup, and the annexes show the turns.*

So these annexes are really a collection of findings and recipes — the sort of material that would normally live in a wider series of technical notes written for engineers (and we may yet spin them out of what has become a fairly bloated tutorial). But we think they're also for anyone curious about how a messy technical problem can be approached systematically. Enjoy!

Annex 1 — How we create reference data for entities

This is the meaty annex, because entity resolution is where reality bites. It's a pipeline narrative, not a command dump: a membership list becomes verified filings, which become a reviewed entity KB, which becomes the chunk labels you saw in Module 2.

A1.1 — The membership list (the scope)

A downloaded corpus's roster is **not** hardcoded — it lives in a **scope file** at the corpus root, `data/corpora/<corpus>/<corpus>_full_scope.yaml`: one row per firm, keyed by a readable `label` and carrying the identifiers the downloader needs. The access key differs by source, but the **identity key is always the LEI**:

- `sec_10k` → rows of `{label, cik, ticker, lei}` — downloaded from **SEC EDGAR** by CIK.
- `esef_banks` → rows keyed by **LEI** — downloaded from **filings.xbrl.org**.

The two corpora that ship in the box are built differently and carry no such roster: **demo** is ingested from documents bundled with AIStudio, and **help** is generated from AIStudio's own documentation. The scope file is how a *downloaded* corpus declares which firms it contains.

Where each source is reached — endpoint, access key, rate limits — is kept as data, not baked into code (`data/knowledge_sources/gleif/gleif_metadata.yaml` carries the base URL and endpoints for GLEIF; EDGAR and filings.xbrl.org are configured the same way). Adding a regulator is configuration, not a rewrite. To add or remove a firm, edit the scope file — never the downloader.

A1.2 — Pulling the filings

The downloader reads a **scope** — a membership list of `{label, cik|ticker}` — and fetches one firm at a time (default: the 5 most recent annual filings), writing into `data/corpora/<corpus>/uploads/`. Module 2's bare `ais_download_sec_10k` was scope-driven all along: it reads the default `data/corpora/sec_10k/sec_10k_full_scope.yaml`. You can also target a single firm, or supply your own list:

```
ais_download_sec_10k                          # default scope (the curated portfolio)
ais_download_sec_10k --cik 0000886982         # one firm, by CIK
ais_download_sec_10k --tkr BNY --years 5      # one firm, by ticker
ais_download_sec_10k --scope my_firms.yaml    # bring your own list (same schema as the d
ais_download_esef --scope lang_en             # ESEF: a language-segmented subset
```

Bring your own list (Beat 2). The default scope is just a file. Copy `sec_10k_full_scope.yaml`, keep the schema (`entities: [{label, cik|ticker}]`), list the firms you want, and run `--scope <your.yaml>`. The "you were lucky" set of Module 2 was lucky precisely because someone already curated that file — every firm in it posts clean iXBRL with the entity tag. Your own list won't be pre-vetted, which is what the next part is about.

Verify at download (Beat 1). With the curated scope every filing carries the tag ingest needs. Off the curated path that's not guaranteed — so the downloader **verifies each filing as it lands**,

reading the same `dei:EntityRegistrantName` tag the ingest pipeline's primary strategy reads, and reports ✅ / ❌ per filing. Worked example — try a firm whose older filings predate the tag:

```
ais_download_sec_10k --tkr BNY --years 5
```

BNY Mellon's older 10-Ks carry `dei:AuditorName` but **not** `dei:EntityRegistrantName` (the same class as Wells Fargo). So verify comes back all-❌ and the downloader coaches:

```
❌ ...: none of the downloaded filings (2021, 2022, 2023, 2024, 2025) carry an iXBRL
   entity tag (dei:EntityRegistrantName), so ingest can't auto-recognize them.
   You can override:
     ais_download_sec_10k --tkr BNY --years 5 --force_name "<name to use>" [--force_year <Y
   But it's worth finding out *why* the filing lacks the tag first — see Tutorial Annex 1.
```

`--force_name` / `--force_year` let you assert the identity the tag didn't provide. This is the **download-time twin of the worksheet's name-less lever** (A1.4 case 3): same fix — the operator supplies the name — done early, at fetch, instead of later, at the worksheet. The assertion is recorded to a sidecar (`sec_10k_entity_overrides.yaml`) so it survives to ingest. But the coaching is sincere: a missing mandatory tag usually means you've got the wrong document or a pre-iXBRL filing, and forcing a name papers over that rather than fixing it.

A1.3 — Recognizing entities (scan → guess → worksheet)

Running `ais_import_entity_kb --corpus <corpus> --apply` does three things:

1. **Scan** every filing in `uploads/` and read its self-reported iXBRL entity name (`dei:EntityRegistrantName`).
2. **Best-guess** each against GLEIF.
3. If everything resolves cleanly, **apply** — write the entity KB. If any row needs a human, the **`full_scope` itself is the worksheet**: there's no separate file — you edit `data/corpora/<corpus>/<corpus>_full_scope.yaml` in place and re-run.

The doctrine, stated plainly: **the machine guess is a starting point; the human-verified LEI is the input; the scope is the trust boundary.** A name search can return a confident, well-formed, *wrong* legal entity — and a clean ingest of the wrong entity is invisible to every downstream check. The scope exists so a human confirms the identity once, before any of that data is trusted.

The scope is an open schema. Every non-`label` field names its own provenance, and the **bare** field is the user-authoritative one that **wins**:

```
- label: BlackRock                  # the handle (user)
  cik: 0002012383                   # bare — the download key (user/seed)
```

```
  ticker: BLK                       # bare (user)
  lei: REPLACE_WITH_VERIFIED_VALUE_IF_AVAILABLE       # bare — YOUR verified LEI; WINS. Sent
  xbrl_name: REPLACE_WITH_VERIFIED_VALUE_IF_AVAILABLE # bare — user name override; sentinel
  sec_xbrl_name: BlackRock, Inc.    # the filing's self-reported tag (source; populated by so
  gleif_lei: 529900VBK42Y5HHRMD23  # GLEIF's confirmed LEI (source)
  files: [CIK_0002012383_10K_2025-02-25.htm]
```

The rule is `<provenance>_<data_type>` for a source's value (`gleif_` , `sec_` , `wikidata_`) and a **bare `<data_type>`** for the human value. Resolution reads **bare → source → empty**: a verified bare `lei` overrides the machine-guessed `gleif_lei` ; a bare `xbrl_name` overrides the filing's `sec_xbrl_name` . The bare field *is* the canonical — there is no separate `canonical` field. Unknown bare fields ship as `REPLACE_WITH_VERIFIED_VALUE_IF_AVAILABLE` , a fill-me marker the resolver treats as empty. This is what makes the schema **open**: a new source is a new prefixed field, and a human correction is always one bare field away — neither touches code.

A1.4 — When reality bites: the four levers

Every row the scan can't resolve falls into one of four cases. Each has one fix in the worksheet:

1. **Resolved-but-verify.** The guess looks right; confirm it. *Example: Jefferies — resolves, but you confirm the registrant collapse is the operating company, not a holding shell.*
2. **Name-resolution failure → fill `lei` .** The name search returned the wrong entity (a subsidiary, a depositary receipt, a namesake). You paste the correct 20-character LEI into `lei` and re-run; the row is rewritten deterministically by LEI. *Examples: AllianceBernstein and Wells Fargo (US); Société Générale (→ US sub), Crédit Agricole (→ Brazilian sub), BBVA (→ Santander ADR) on the European set.*
3. **Name-extraction failure / name-less → author the bare `xbrl_name` .** The filing's entity tag is missing or mangled (e.g. inline-markup whitespace collapse → "Erste GroupBankAG"), so there's no name to resolve. You author the entity name in the bare `xbrl_name` field. *Example: BNY Mellon and the whitespace-collapse class.* This is the same assertion `--force_name` makes at download time (A1.2) — the scope is the later, canonical place to make it.
4. **Wrong/missing entity at ingest → the `expected_xbrl_name` guard.** A filing whose self-reported entity doesn't match the target's `expected_xbrl_name` is **skipped unless `--force`** — so a mis-mapped filing can't silently ingest under the wrong firm. *Example: the CBOE filings that were actually Larimar Therapeutics' 10-K under a wrong CIK — a clean ingest of the wrong company, caught only by this guard.*

The first time through, you fill `lei` for the failures, re-run, and the table goes clean. From then on the edits persist across re-runs — you review once.

A1.4b — The scope's open/provenance field convention — and why LEI is the shared key

The `full_scope` file is the single source of truth for corpus membership. Each entry is keyed by `label`; every other field encodes two dimensions:

```
<provenance>_<data_type>   — a source's reported value (sec, gleif, wikidata, esef, ais)
<data_type>                — bare = the user/by-hand value, which is AUTHORITATIVE and wins
```

The **bare field IS the canonical** — there is no separate `canonical` field. A source's guess lands in its prefixed field; you fill the bare field only to override a bad guess. **Resolution rule:** bare (user) → golden source(s) → empty.

data_type	bare (user, wins)	source field(s)
`lei`	`lei` — operator-typed LEI; **the shared key, used thereafter**	`gleif_lei`
name	`xbrl_name` — user-authored name	`sec_xbrl_name` / `esef_xbrl_name`, `gleif_name`
`cik` / `ticker`	`cik` / `ticker`	`sec_cik` / `sec_ticker`
`jurisdiction` / `filing_language`	bare	`sec_*` / `esef_*`
`aliases`	`aliases`	`gleif_aliases` ∪ `wikidata_aliases` → `combined_aliases`
`expected_xbrl_name`	seed — download-verify guard (fetched filing must match)	—
files / last_updated	tool-managed	`ais_files` / `ais_last_updated`

For LEI specifically: `gleif_lei` holds what GLEIF confirmed; if you find a better one, fill the bare `lei` and it wins. There is **no `lei_corrected`** — vestigial, replaced by this convention.

This is why the 2026-06-16 ESEF build succeeded despite five firms filing under names we didn't expect — Nordeakoncernen, Erste Group BankAG, Barclays Bank PLC, StandardChartered Bank, Skandinaviska Enskilda Banken AB (publ). The LEI resolved each

identity correctly regardless of what the filer typed. For European entities, **supply the LEI — the rest resolves from there.**

> ***Implementation status.*** *This open/provenance convention is the decided target, ratified on the documentation side. It is* ***not yet wired****: the live resolver (* `_scope_common_ops.py` *) still uses the old cascade (* `lei_corrected → gleif_lei → lei` *,* `canonical → gleif_canonical → xbrl_name → label` *), and on-disk rows carry old names. The resolver rewire + one-time file migration are parked for PIPELINE.*

A1.5 — Correcting a LEI: a worked example

When a row resolves by name-search, the review prints it as `(name-search, not verified LEI)` — the LEI is a machine guess sitting in `gleif_lei`, and the bare `lei` is still the sentinel. Promoting it to verified is one edit. BlackRock is the live case: added on demand, it name-searched to `529900VBK42Y5HHRMD23` — the post-2024-reorg successor (previous legal name *BlackRock Funding, Inc.*, confirmed at search.gleif.org). The guess was right, but it's still only a guess until a human says so.

1. Open the scope in an editor: `bash open -e ~/Developer/AIStudio/data/corpora/sec_10k/sec_10k_full_scope.yaml`
2. Find the row (here `label: CIK_0002012383`) and set the **bare** `lei` to the value you verified at GLEIF, replacing the sentinel: `yaml lei: 529900VBK42Y5HHRMD23`
3. Re-resolve and rebuild — **no re-ingest** (chunks are untouched): `bash ais_import_entity_kb --corpus sec_10k # now resolves BY your verified LEI ais_import_entity_kb --corpus sec_10k --apply # rebuild the KB ais_start # reload the cached KB`

The review line flips from `(name-search, not verified LEI)` to a clean resolve: the bare `lei` you typed now **wins** over `gleif_lei`, so the row is verified by a human, deterministically. That precedence — the bare user value over the machine guess — is the whole point of the open schema.

A1.6 — Apply, and what you get

When the table is clean, `--apply` writes `data/knowledge_sources/gleif/gleif_<corpus>_full_entities.yaml` (schema 1.1):

```
schema_version: '1.1'
source_id: gleif
corpus: <corpus>
scope_id: full
```

```
entity_count: <N>
entities:
  - canonical: <GLEIF canonical name>
    scope_name: <the natural query form — what users type>
    lei: <20-char LEI>
    aliases: [<short names, tickers, case variants>]
    wikidata_label: <optional>
    wikidata_short_name: <optional>
    wikidata_tickers: [<optional>]
```

Each firm is **one entry in the `entities:` list**, identified by its **LEI**. At ingestion the pipeline reads the chunk's self-reported entity, matches it against this list, and prefixes the chunk `[Document: <canonical> | <scope_name> | <ticker> FY<year>]` — so the firm and year travel inside the chunk text. The LEI stays in the KB as the **identity key**; it is deliberately **not** in the prefix (it isn't a retrieval token and would only add noise to keyword matching). `scope_name` is the human-facing name the prefix and queries use; `aliases` widens lexical recall.

A1.7 — The European parallel

Module 3 runs this exact flow with the LEI as the *download* key (filings.xbrl.org filters by LEI prefix). The headline lesson lives here: on the European set a name search confidently mismatched roughly half the firms — the strongest possible argument for LEI-is-input. The worked `lei` corrections for the European banks are the canonical example of the second lever above.

Annex 2 — How we create reference data for the glossary

Deliberately short — it's the contrast to Annex 1. The glossary is the *symmetric, simple* reference layer: no API to fight, no entity to disambiguate, no review gate.

A2.1 — What it is

A glossary maps a domain term to an expansion string. It is applied **at query time only** — chunks are never rewritten. This is the asymmetry that makes it the opposite of the entity KB (which is baked into chunks at ingestion and used as a retrieval filter). Understanding the contrast is the teaching point: two reference layers, one human-gated and build-time, one curated and query-time.

A2.2 — The curated source

The bank corpora use **`bis_basel`** — Basel III/IV regulatory vocabulary, a static, hand-curated source. Because it's deterministic, there is **no review gate and one pass** — nothing to resolve against a flaky registry, nothing for a human to confirm. (Contrast: the BIS SCO95 source page renders via JavaScript and isn't scrapeable, which is exactly why it's curated rather than fetched.)

```
ais_import_glossary_kb --source bis_basel
```

writes `data/knowledge_sources/bis_basel/bis_basel_<corpus>_<scope>_glossary.yaml`. Each entry maps a term to its full form and a keyword expansion string:

```
glossary:
  - term: CET1
    full_form: Common Equity Tier 1
    expansion: CET1 Common Equity Tier 1 capital ratio regulatory capital
  - term: RWA
    full_form: Risk-Weighted Assets
    expansion: RWA risk-weighted assets capital requirements standardized approach
```

A2.3 — How it's used

At query time, AIStudio looks up each glossary term in your question and widens the keyword (BM25) search to include its expansion — so *"CET1"* reaches *"Common Equity Tier 1 capital ratio regulatory capital"* and a question phrased either way finds a passage phrased the other way.

A2.4 — Extending it

`ais_import_glossary_kb` takes `--source`, `--corpus`, and `--scope`. Additional sources (`nist_ai_rmf`, `esrs`) are declared but not yet built. A new glossary source is a new curated seed plus one import run — no worksheet, no review.

The spine across both annexes: two reference layers — **entity** (build-time tag + query filter, human-gated, asymmetric) and **glossary** (query-time expansion, curated, symmetric). Annex 1 earns its length because the data fights back; Annex 2 is short because it doesn't.

Annex 3 — When the portfolio isn't in English

A US-only corpus hides a problem that the European set surfaces on the first query: **retrieval quality is not language-neutral.** Everything in Modules 2–3 — embeddings, BM25, the glossary, the score threshold — was tuned against English text, and each one degrades differently when the filing isn't in English. This annex is the honest account of where the language ceiling is, what we can do about it, and what we can't.

A3.1 — Why a European portfolio is harder than a US one

Two things change at once when you cross the Atlantic, and they compound:

- **The filings aren't all in English.** ESEF filers report in their primary language — French, German, Dutch, Norwegian — sometimes with an English version, sometimes not. A question in English against a filing written in French is a cross-language retrieval problem the US corpus never poses.
- **The legal structure is messier.** European groups carry more subsidiaries, depositary receipts, and namesakes — which is why entity resolution (Annex 1) fails far more often here. A name search for these banks confidently mismatched roughly half the set: Société Générale → a US subsidiary, Crédit Agricole → a Brazilian subsidiary, BBVA → a Santander depositary receipt. That's an *identity* problem (Annex 1's `lei` fix), but it has a *language* root — the jurisdiction default and the multilingual registry are what make the guess wrong.

The observable result: the European corpus runs materially below the US one on objective quality even after the entity work — the gap is the language ceiling. We deliberately don't reduce it to a single percentage: the 17-firm set isn't a random sample, so a headline number would imply more precision than the observation supports.

A3.2 — The three places language degrades retrieval

1. Terminology mismatch — the glossary is English-anchored. Ask about *"CET1"* and a French filing may only ever say *"fonds propres de base de catégorie 1."* The BIS Basel glossary (Annex 2) expands the *English* acronym to its English full form — it does **not** translate. So the glossary closes the acronym↔full-form gap *within* English but does nothing for the English↔French gap. Cross-language terminology is an open seam, not a solved one.

2. Mislabeled `filing_language`. A corpus row that claims a filing is English when it isn't poisons everything downstream — scope filtering, any language-conditioned expansion, the operator's mental model of the set. Worked example: **DNB** was carried as `filing_language: en` when the filings are Norwegian; the fix is to correct the label and tighten the `lang_en` scope

(12 EN-primary firms, not all 17). The lesson generalizes: the language tag is reference data like the LEI — verify it, don't trust the downloaded default.

3. Accent and diacritic mismatch. A query token without diacritics (`Credit Agricole`) and a filing token with them (`Crédit Agricole`) are different strings to a naive matcher, and the accented form can score **below the relevance threshold** and drop out before the model ever sees it. Worked example: **Crédit Agricole**'s French terminology fell under `min_score` on an early run — a retrieval miss caused purely by orthography. The mitigation is accent normalization at query and index time (folded into the harness), but normalization is a patch on a tokenizer that wasn't built for the problem.

A3.3 — The levers we actually have

- **`lang_*` scope segmentation.** The scope taxonomy (`lang_en`, `lang_fr`, `lang_en_fr`, `lang_not_en`) lets you benchmark and tune a language-coherent subset instead of averaging across a mixed set, so a French-filing failure doesn't hide inside an English-majority score. `esef_banks_lang_en_scope.yaml` (the EN-primary subset) + its basic question set are the canonical instance.
- **A bigger model buys multilingual headroom.** On the same European questions, the larger synthesis model showed better citation discipline and visibly better multilingual handling than the small one — at a latency cost. This is one of the few language levers that helps across the board rather than patching one failure mode.
- **α tuning is not the fix — entity isolation is.** Pushing the retrieval mix toward keyword (α≈0.75) fixed one firm's dominance (ING) but immediately introduced another's (Santander) — squeezing the balloon. The durable fix for "one firm's chunks crowd out another's" is the per-firm retrieval quota and the entity filter (Module 2's chunk labels), not a global α knob. α is a per-query convenience, not a corpus-level cure.

A3.4 — The honest limits

This is a **frontier, not a closed problem.** Cross-language terminology has no glossary bridge yet; non-Latin and heavily-accented text still stresses the tokenizer; and a multilingual corpus's objective-% sits structurally below an English one until those are addressed. The right operator posture is the same as everywhere else in AIStudio: segment by language so the failures are visible, verify the language labels like any other reference data, and read the cited chunks (Annex 4) rather than trusting a score that English tuning inflated.

Annex 4 — Benchmarking

`ais_bench` is easy to run and easy to misread. This annex is the story of how we learned to read it — because the naive score lies in a specific, dangerous direction, and the whole audit discipline exists to catch that lie.

A4.0 — A benchmark run is a coordinate, not just "the questions"

A run isn't simply *ask the questions*. It evaluates one point in a space of choices — **which questions × which firms (the scope) × the retrieval settings × how entities are handled × which model** — scored by a grading method. Reproducibility means writing that coordinate down: the question file and the scope file *are* the experiment record, and the run flags are its conditions. That is why the tests live in files, not in code.

The settings that move the result, and why each sits where it does:

- **Top K = 10** for multi-firm financial corpora — fewer slots drop firms from a comparison; 5 is fine for small single-topic corpora.
- **Retrieval Mix (α) = 0.5.** Pure conceptual retrieval loses the *named firm*: the query embedding is dominated by the shared concept (every bank discusses "CET1"), so the firm names barely move it and retrieval latches onto whoever wrote most densely about the topic. Blending in literal matching restores the names. 0.5 is the locked default.
- **Score Threshold (min_score).** A relevance floor that drops weak chunks — but set too high it starves genuinely relevant ones (the embedding model under-scores them), so ~0.2 is the working value for these corpora.
- **Entity handling — isolation, not expansion.** The central lesson: recognizing a firm and *appending* its name to the query does **not** stop the wrong firms' chunks from being retrieved. Only a retrieval-time **filter** keyed to the recognized firm isolates correctly — and AIStudio builds that filter automatically. Expansion decorates the query; the filter excludes the contaminants. Conflating the two once produced answers that cited the wrong companies while scoring 100% mechanically.
- **Model.** When retrieval is the bottleneck, model size barely moves the score — a small local model matches a large one on retrieval-limited questions; the larger model pulls ahead only on dense multi-firm synthesis. Either way, a model handed a chunk that lacks the fact will *fabricate* rather than abstain — which is exactly why the audit reads the cited chunk, not the answer's confidence.

One structural ceiling sits above all of this: **language.** On non-English filings accuracy falls off by language (English near-perfect, French partial, others poor) — an embedding-and-extraction limit, not an isolation failure — so it is measured separately rather than hidden inside the score.

A4.1 — What the mechanical score actually checks

A benchmark question carries `keywords` and an expected answer shape. A run marks a question **pass** on three checks, all mechanical:

1. every `keyword` appears in the answer,
2. the answer carries at least one citation,
3. the model didn't hedge with "no information available."

That's it. Notice what's **not** in the list: whether the answer is *correct*, whether it cites the *right firm*, whether the cited chunk actually *supports* the claim. The mechanical score is a presence test, not a truth test.

A4.2 — The trap: a passing question can be wrong

Because the checks are presence tests, a question can pass all three while being substantively wrong. Real cases from the record:

- A KBC net-interest-income question **passed** — keywords present, citation present — while the answer was entirely about **ING**. Right keywords, wrong firm.
- A multi-firm question about JPMorgan / Bank of America / Citigroup **passed** while citing **Prudential Financial**. Right shape, wrong source.

The keyword check is a *signal*, not a guarantee of source correctness. This is the single most important thing to internalize about the harness: **mechanical pass ≠ quality.** We measured a no-scaffold run at **8/8 mechanical and 1/7 audited** — a 100% mechanical score that was almost entirely wrong on inspection.

A4.3 — The worst case: fabrication scores *higher* than honesty

The trap gets actively perverse when you compare models. On the same seven SEC questions, a fast small model (mistral) ran 3× quicker and **fabricated** — an incoherent CET1, a revenue figure off by 10× ("$1,618 billion"), citations mapped to the wrong firms' filings, invented disclosure dates — while the standing model (gemma3:27b) correctly **abstained** where retrieval starved it. The mechanical score: **mistral 7/7, gemma 6/7.** The keyword-and-citation gate *rewarded* the confident fabrication and *penalized* the honest abstention.

That's the strongest possible argument that the mechanical verdict, used alone, optimizes for exactly the wrong behavior. Two standing consequences:

- **mistral is disqualified as an unsupervised extractor** for financial data; gemma3:27b is the benchmark/synthesis model and we don't switch off it.

- gemma fabricates too **when retrieval starves it** — so the **abstention guardrail** (the model must say "I don't have it" rather than invent) matters for every model, and a fabrication-catching verdict matters more than a faster one.

A4.4 — The fix to the verdict

We stopped trusting the pass-rate and moved the verdict to signals that catch fabrication:

- **Objective-%, not "pass-rate."** Define the quality metric as `a / (a + b + c)` — the fraction of answers that are *objectively correct* (right content, right firm, supported by the cited chunk), excluding architectural ceilings. We call it "objective %," deliberately not "honest %" — *honesty is a property of the measurement discipline, not a moral claim about the model.*
- **A four-state audit, not a binary.** Every audited question gets one of: ✅ **Good** (correct, right firm, substantive, cited), ⚠ **Partial** (mechanically passing but incomplete or wrong firm), ❌ **Miss** (retrieval or generation failure), 🔍 **Grading artifact** (mechanically failed but substantively *correct* — keyword brittleness, citation dropout, an accent mismatch). The grading-artifact bucket is the mirror image of A4.2: it's where the mechanical score *understates* quality, and it's what tells you to fix a keyword list rather than the engine.
- **An amber, entity-weighted score.** The weighted-sum grader leads with **entity coverage** (did the answer cite the firms it should) and demotes raw keyword presence to one signal among several — so confident-but-wrong-firm answers can't score green.

A4.5 — The discipline that makes it real: verify the cited chunk

None of the above works on trust. A confident, well-cited, fluent answer can still be fabricated — the only thing that confirms a claim is **scrolling the cited chunk** and reading whether it actually says what the answer claims. Worked example: a CET1 answer that *looked* right was only validated by pulling the exact Qdrant chunk and confirming the number and the column it came from. The rule — **verify-the-artifact**, or here, *verify-the-cited-chunk* — is why an audit is a read, not a re-run. A re-ingest that "completed successfully" is a claim; the chunk is the evidence.

A4.6 — Running it, and the canonical settings

```
ais_bench --corpus sec_10k --top-k 10
```

writes a timestamped report to `benchmarks/<corpus>/reports/` in three formats (`.md` readable with answers, `.json` machine-readable, `.pdf` if `weasyprint` is installed). The harness reads the right parameters per corpus from metadata, so you rarely pass flags by hand. Canonical configuration: **α = 0.5, K = 10** for `sec_10k` and `esef_banks`; **K = 5** for `demo` / `help`. Per-question `entity_filter` lives in the question YAML and is passed through to retrieval.

The one-line takeaway for an operator: the green number is where you *start* reading, not where you stop. Run the bench for the signal, then audit the answers — read the cited chunks, check the firms — because the mechanical score's failure mode is to reward exactly the confident wrongness you most need to catch.

A4.7 — Naming scopes so the benchmark binds them

> ***Tip — name scopes so the benchmark binds them automatically.*** *A scope file named* `<corpus>_<description>_scope.yaml` *lets you run* `ais_bench --corpus <corpus> --scope <description>` *without repeating the corpus name or a path —* `ais_bench` *resolves* `<corpus>_<description>_scope.yaml` *from the convention. For example, a file* `sec_10k_big_banks_scope.yaml` *is reached with* `ais_bench --corpus sec_10k --scope big_banks`*. The same naming works for the download scope (* `--scope` *), so one consistently-named file serves both. Keep your corpus's scope, questions, and metadata files together under* `data/corpora/<corpus>/`*.*

A4.8 — Two question sets, and what running both teaches

A benchmark is only as honest as its questions (A4.0). To see that directly, the `sec_10k` corpus carries two question sets worth running back to back:

- **The default set** — questions shaped to what the system handles cleanly today: narrative comparisons (risk disclosure, cybersecurity, climate, strategy), single-firm descriptions, and single-figure lookups. Run it with `ais_bench --corpus sec_10k`.
- **A harder, dated set** — the same topics pushed into the area still under development: multi-year, multi-firm *quantitative tables* (a five-year capital-ratio-and-revenue trend across three banks in a single question). Run it by pointing `--questions` at the `..._June_2026_...` file.

Run both and the contrast is clear and repeatable. The narrative and single-fact questions come back clean — right firms, grounded answers, correct citations, no cross-firm bleed. The multi-firm *table* question is where the work continues: the system reliably retrieves the right firm's filing, but lifting an exact figure out of a dense, multi-column capital table — binding each number to its row label and its year — is **work in progress**, and it will sometimes return a confident value drawn from an adjacent row or year.

The takeaway is about method, not a verdict on the tool: **which question shapes a retrieval system handles well is itself a finding.** Narrative synthesis and firm isolation are solid here; precise table-cell extraction is the active frontier (Annex 5 walks one such case in detail).

Running both sets is how you keep that boundary visible and stated up front — rather than discovering it live in front of someone.

Annex 5 — The Table Problem (Ahhh… tables…)

Everything to this point assumed the answer lives in a *sentence*. Financial filings break that assumption constantly: the number you want sits in a cell, and a cell only means something through its **column header** (the period — *FY2025* vs *FY2024*) and its **row header** (the variant — *Standardized* vs *Advanced*). RAG chunks a document into ~1,200-character windows; when a chunk boundary falls between the header band and the data row — or when the model reads across a wide row to the wrong column — the cell arrives stripped of the headers that gave it meaning. The model then answers fluently and **confidently wrong**. This annex is one worked case where the failure and the success sit side by side, because the contrast *is* the teaching point.

A5.1 — The query that exposes it

Ask the SEC corpus a cross-firm numeric question:

> *"Compare the CET1 ratios for JPMorgan and Citigroup."*

Grounded in each firm's FY2025 10-K, AIStudio returns JPMorgan **≈ 15.7%** and Citigroup **13.2%**, both labeled *as of December 31, 2025*, each citing the correct firm's filing. Entity isolation worked — right firms, right documents, no cross-contamination. **But one of those two numbers is exactly right and the other is a full year wrong** — and nothing in the answer tells you which.

A5.2 — Why Citigroup is right and JPMorgan is wrong: prose vs. grid

The difference is *how each filing states the number*, not which firm the system prefers.

- **Citigroup states it in a sentence.** The Citi 10-K's Capital section says, in prose, that the CET1 ratio was **13.2%** as of December 31, 2025, compared with **13.6%** a year earlier, on the Standardized approach. A sentence keeps the value bonded to its period and its basis — so it survives chunking intact. AIStudio reproduced all of it (the 13.2%, the 13.6% prior year, the Standardized-is-binding point) faithfully.
- **JPMorgan states it in a multi-year column table.** JPM's CET1 row reads, across columns, **14.6% (2025) · 15.7% (2024) · …**. The true *as-of-December-31-2025* figure is **14.6%**. AIStudio returned **15.7%** — the **2024 column** — and labeled it 2025. Right row,

wrong column. (Confirmed against the primary filing and JPM's own February-2026 disclosure of a 14.6% current Standardized ratio.)

That is column-header detachment in its purest form: the prose figure survives, the grid figure loses the year and lands one column off.

A5.3 — The second loss: the row header (and the tell it leaves)

The same mechanism corrupts the *variant* (the row header) — and it leaves a tell you can catch without ever opening the source. Both firms came back with an "Advanced" figure that is **structurally impossible**:

- AIStudio gave JPM an Advanced ratio (≈15.8%) **higher** than its Standardized — but JPM's 10-K states that as of December 31, 2025 the *Advanced* approach became the **more binding** (i.e., lower) one. Advanced above Standardized cannot be right.
- AIStudio gave Citi an Advanced ratio (≈11.9%) **below** its Standardized 13.2% — yet the same answer correctly said Citi's *binding* ratio is the Standardized one, which is only true if Advanced is the *higher* (non-binding) number. An Advanced below the binding Standardized is self-contradictory.

Catching this needs no source at all: each answer **contradicts itself**. That is the operator's cheapest table-misbind detector — when a derived comparison violates a rule the answer itself states (binding = the lower of the two), suspect a row/column detachment, not a real datum.

A5.4 — Why it's hard, and what AIStudio does about it

A naïve text extractor flattens a table to a stream of numbers and the headers dissolve. AIStudio's ingestion runs a table-extraction normalizer (`loaders.py` / `chunking.py`) that detects grids and serializes them to header-bonded markdown rows, so a clean single-header table survives as `| CET1 capital ratio | 13.1 | % | … |` with the label attached — that closes the common case. The open frontier — the "hard 10%" — is exactly what the JPM example hits: **stacked / multi-level column headers** (a year band sitting *over* a Standardized/Advanced band) and **chunk boundaries that sever the header band from the data rows**. Composing multi-level headers into each data cell (`label (Standardized, FY2025): value`) is the header-binding work tracked as the table-understanding item; until it lands, multi-year multi-basis tables are the residual risk.

A5.5 — The operator's defenses, today

- **Name the column in the query.** Asking for *"JPMorgan's consolidated holding-company Standardized CET1 ratio as of fiscal year-end 2025"* hands the retriever and the model the column and row keys explicitly — in the record, re-asking a misread CET1 question *with*

the column named returned the correct cell from the very same chunk. The data was always there; the query supplied the missing header.

- **Verify the cited chunk — including the column.** This is Annex 4's discipline (A4.5) applied to tables: scroll the cited chunk and confirm not just that the number appears, but that it sits under the period and basis the answer claims. A table answer is verified at the *cell*, not the page.
- **Trust prose over grids, and cross-check the period.** A figure stated in a sentence can be weighted; a figure lifted from a table should be treated as needing confirmation. A two-firm comparison where one number is prose-sourced and the other grid-sourced — this exact case — is the highest-risk shape there is.

A5.6 — The honest limit

Prose-stated numbers extract faithfully; **table-grid numbers are a frontier, not a solved problem** — single-header grids are handled, stacked-header multi-year tables are not yet. The posture is the one the whole tutorial keeps returning to: read the cited chunk, distrust a confident derived comparison that contradicts itself, and treat a table figure as a claim to verify rather than a fact to repeat. The system's job is to make the misbind *visible* — the self-contradiction in A5.3 is that visibility — and closing it for good is the header-binding serializer's job.

Annex 6 — Under the Hood: Where Data Lives

The modules keep file paths out of the way on purpose. This annex is the map — where AIStudio stores each piece on disk, so you can find, back up, or inspect it. Everything sits under the AIStudio install directory.

What	Where	Notes
Source documents	`data/corpora/<corpus>/uploads/`	The ingested filings, or your own uploaded files.
Membership manifest	`data/corpora/<corpus>/<corpus>_full_scope.yaml`	The `{label, cik|lei}` rows driving download + entity resolution (**Annex 1**).
	`data/corpora/<corpus>/<corpus>_corpus_metadata.yaml`	Query defaults + ingest counts. Edit

What	Where	Notes
Corpus settings + stats		via the UI **Edit** modal.
Entity knowledge base	`data/knowledge_sources/gleif/<corpus>_full_entities.yaml`	Canonical name + LEI + aliases per firm (**Annex 1**).
Glossary knowledge base	`data/knowledge_sources/bis_basel/bis_basel_<corpus>_<scope>_glossary.yaml`	Term → full form → keyword expansion (**Annex 2**).
Source metadata	`data/knowledge_sources/gleif/gleif_metadata.yaml`	Endpoints, base URL, rate limits — kept as data, not code.
Benchmark questions	`benchmarks/<corpus>/<corpus>_questions.yaml`	The benchmark suite (**Annex 4**); reports in `reports/`.
The vector index	Qdrant collection `aistudio_<corpus>`	The embedded chunks; rebuilt on every ingest (`--force` wipes first).

The shape of it: everything *about* one corpus lives under `data/corpora/<corpus>/`; the *shared* reference data — the entity and glossary knowledge bases, and source metadata — lives under `data/knowledge_sources/`; and the vector index lives in Qdrant. The source documents and the two YAML knowledge bases are the human-editable inputs; the metadata file's ingest stats and the Qdrant collection are machine-written outputs.

Reference — Commands

Command	What it does
`ais_start`	Start all services and open the UI
`ais_stop`	Stop all services

Command	What it does
`ais_download_sec_10k`	Download SEC 10-K filings (reads the corpus scope file)
`ais_download_esef`	Download European ESEF reports (reads the scope file, by LEI)
`ais_import_entity_kb --corpus <name> --apply`	Build a corpus's entity KB (resolves firms by verified LEI)
`ais_import_glossary_kb --source bis_basel`	Build/refresh the domain glossary (ships ready-built)
`ais_ingest_sec_10k` / `ais_ingest_esef`	Ingest a corpus (~30 min); applies chunk enrichment. Incremental by default; `--files <patterns>` ingests only matching files (comma-separated substrings/regexes, OR-matched); `--force` rebuilds
`ais_bench --corpus <name>`	Run the benchmark suite for a corpus
`ais_log` · `ais_help` · `ais_install`	Tail log · command reference · install/update

Reference — Sources & Lookups

When you build your own scope (Module 4, Annex 1) you'll need to look up identifiers by hand. These are the authoritative, free sources for each:

You need	Where to look it up	Source
CIK (US filer ID) and the filings themselves	Company/CIK lookup — `https://www.sec.gov/search-filings/cik-lookup` · full-text search — `https://www.sec.gov/edgar/search/`	**SEC EDGAR** (U.S. Securities and Exchange Commission), `https://www.sec.gov/edgar`; data API `https://data.sec.gov`
Ticker → CIK (the map AIStudio resolves `--tkr` against)	`https://www.sec.gov/files/company_tickers.json`	**SEC EDGAR**
LEI (the 20-char legal entity ID — the entity-KB input)	LEI search — `https://search.gleif.org` (US alt: OFR finder `https://www.financialresearch.gov/data/legal-entity-identifier/find-lei/`)	**GLEIF** (Global Legal Entity Identifier Foundation), `https://www.gleif.org`; API `https://api.gleif.org/api/v1`
	`https://filings.xbrl.org`	

You need	Where to look it up	Source
European (ESEF) filings, by LEI		**filings.xbrl.org** (operated by XBRL International)
Basel terms behind the glossary	Basel Framework — `https://www.bis.org/basel_framework/`	**BIS** (Bank for International Settlements)
The **ESEF mandate** itself (why European filings are iXBRL)	`https://www.esma.europa.eu`	**ESMA** (European Securities and Markets Authority)

Practical order for adding a US firm to a scope: find it on **EDGAR** to confirm it files a domestic 10-K and grab its **CIK**; cross-check the **ticker** in `company_tickers.json`; look up the **LEI** on **GLEIF** for the `lei` field (used by entity resolution and the benchmark scope). For a European bank, the **LEI** is the primary key — start at **GLEIF**, then confirm filings exist on **filings.xbrl.org**.

> *Identifiers are reference data: verify them at the source, don't trust a guess. A wrong CIK silently downloads the wrong company's filing — and EDGAR is the only authority that settles it.*